Retail Inventory & Purchase Order System

GROUP 4 | Mudavath Tharun, Kuldeep Choudhary, Shubham Shekhawat | Team size: 3 | Duration: 8 days

Backend	Frontend	Database	Auth	Deployment
Flask (REST API)	React + Vite	PostgreSQL + SQLAlchemy	JWT (access + refresh)	Docker Compose

1. Problem Statement

A small retail store tracks stock, raises purchase orders to suppliers, receives goods (sometimes partially), and records sales - with a stock figure that nobody can edit by hand.

2. Roles & Permissions

Role	What they can do
Owner	Full access, approves purchase orders, sees valuation and reports.
Manager	Manages products and suppliers, raises and receives purchase orders.
Staff	Records sales and views stock levels only.

3. Data Model

- **User** - credentials, role.
- **Category**, **Product** - SKU, name, category, unit, cost price, selling price, reorder level.
- **Supplier** - name, contact, address, payment terms.
- **PurchaseOrder** + **POLineItem** - supplier, status (draft / approved / partially received / received / cancelled), expected date, ordered qty, received qty.
- **StockMovement** - product, signed quantity, reason (purchase / sale / adjustment / return), reference id, created_by, created_at.
- **Sale** + **SaleItem** - bill number, items, quantities, price at sale, total.

4. Functional Requirements

- Product catalogue with SKU search, category filter and reorder-level highlighting.
- Supplier CRUD with purchase history per supplier.
- Raise a purchase order, approve it, then receive goods - full or partial receipts allowed.
- Sales entry screen that validates stock before saving the bill.
- Reports: low stock, stock valuation at cost, movement history per product, sales by day.
- Manual stock adjustment with a mandatory reason, recorded as a movement.

5. Core Technical Challenge

Stock is a ledger, not a column. Current stock for a product is derived by summing signed `StockMovement` rows; there is no editable `quantity_in_hand` field anywhere. Every receipt, sale, return and adjustment writes a movement row inside the same transaction as its parent document. A sale that would drive stock below zero is rejected with 409. Receiving more than the ordered quantity is rejected. Prove the ledger balances with a report that recomputes stock from movements.

6. React Screens

- Product table with search, category filter and low-stock badges.
- Purchase order wizard: pick supplier, add line items, submit, approve.
- Receive goods screen showing ordered versus already received per line.
- POS-style sale entry with running bill total.
- Dashboard: low stock list, valuation, sales trend.

7. Authentication & Authorisation Specification

- Passwords hashed with bcrypt - plain text passwords must never touch the database.
- Endpoints: `POST /auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`.
- The access token carries a `role` claim; a custom decorator `@role_required("admin")` guards privileged routes on top of `@jwt_required()`.
- Refresh rotation: a refresh call issues a new access token; logout adds the token JTI to a blocklist so it cannot be reused.
- React side: Axios request interceptor attaches the token, response interceptor catches 401, calls refresh once, retries the original request, and logs the user out if refresh also fails.
- Route protection in React by role - a seeker must never be able to open an admin screen, even by typing the URL.

8. Docker Specification

- `backend/Dockerfile` - Python slim base, requirements installed first for layer caching, served by `gunicorn` (not the Flask dev server).
- `frontend/Dockerfile` - multi-stage: Node stage runs `npm run build`, nginx stage serves the static build and proxies `/api` to the backend.
- `docker-compose.yml` - services `api`, `web`, `db` on one network, a named volume for database data, a healthcheck on `db`, and `depends_on` so the API waits for it.
- All secrets (DB URL, JWT secret key) come from `.env`; `.env.example` is committed, `.env` is not.
- Acceptance test: on a clean machine, `git clone` then `docker compose up --build` gives a working app with seeded demo data.

9. Phase-wise Execution Plan

Phase 1 - Day 1: Foundation

- Finalise the data model and draw the ER diagram / collection diagram before writing code.
- Repo and folder structure: `backend/`, `frontend/`, `docker-compose.yml`, `README.md`.
- Docker skeleton up early - database container running and reachable from the API container on day one.
- User model + full JWT auth flow + role seeding. Do not move on until login and refresh work in Postman.

Phase 2 - Day 2 to 3: Core API

- Every core CRUD endpoint with request validation and clear error responses (400 / 401 / 403 / 404 / 409).
- Role guards on every write route; ownership checks so a user can only touch their own records.
- Pagination, filtering and sorting on all list endpoints - never return the whole table.
- Test every endpoint in Postman and save the collection before you write a single React component.

Phase 3 - Day 3 to 5: React Application

- Routing, auth context, protected route wrapper, login and register screens.
- List, detail and create/edit screens for the main entities with client-side form validation.
- Loading, empty and error states on every screen - no blank page while data is fetching.
- Shared Axios instance; no API URL hardcoded in components.

Phase 4 - Day 5 to 6: Core Business Logic

- Implement the core rule described above - this is what separates this project from a CRUD demo.
- Dashboard and aggregation endpoints computed in the database layer, not in JavaScript.
- Search and filters wired end to end.

Phase 5 - Day 6 to 8: Harden and Ship

- Full dockerization verified from a clean clone; fix anything that only worked on your laptop.
- Seed script that creates demo users for each role plus realistic sample data.
- README: architecture overview, setup steps, environment variables, API endpoint table, demo credentials.
- Postman collection, bug fixing pass, and a 5-minute demo recording walking through each role.

10. Work Split

Owner	Responsibility
Member 1	Auth, roles, products, categories, suppliers.
Member 2	Purchase orders, receiving, stock movement ledger, sales, reports.
Member 3	Entire React application.

Every member must commit backend code and review at least one pull request - no single member owns the whole repository.

11. Deliverables

- GitHub repository containing both applications with a readable commit history from every member.
- README.md with setup instructions, environment variables and an API endpoint table.
- ER diagram (SQL) or collection diagram (MongoDB).
- Seed script with demo data and demo credentials per role.
- Postman collection covering every endpoint.
- Working `docker compose up --build` on a clean machine.
- 5-minute demo video.

12. Evaluation

Area	Weight	What is checked
Functionality	30%	All listed features work end to end without manual database edits
API design & JWT	20%	Correct status codes, validation, role guards, refresh and blocklist working
Core business logic	20%	The project-specific core rule is correctly and safely implemented
React frontend	15%	Component structure, state handling, validation, loading and error states
Docker & docs	15%	One-command startup, clean compose file, complete README and seed data

13. Stretch Goals (only after everything above works)

- Barcode/SKU scan input on the sales screen.
- CSV export of the valuation report.